Université Sultan Moulay Slimane
Ecole Supérieure de Technologie
- Beni Mellal -

STRUCTURES DE DONNÉES

Chapitre 2 : Listes Simplement Chainées

Filière : DUT GI
Semestre : S3
Année Universitaire : 2026/2027

Pr. M. OUTANOUTE

Le mur du Chapitre 1

Piles et files du Chapitre 1 : un tableau à taille fixe, MAX. « File pleine » alors que la mémoire de l'ordinateur, elle, est loin d'être saturée — un gaspillage structurel.

Et si la structure pouvait grandir et rétrécir exactement à la taille des données réelles — sans jamais fixer MAX à l'avance ?

La liste chaînée : chaque élément est alloué individuellement, relié au suivant par un pointeur.

Deux acquis sont mobilisés ici

Rappel — (les structures)

typedef struct { ... } est déjà connu.

La nouveauté ici n'est pas la déclaration — c'est qu'une structure va se contenir elle-même, par pointeur.

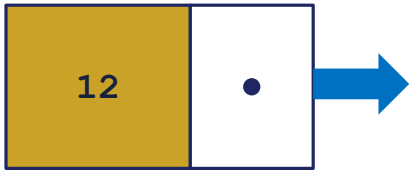
Rappel — (allocation bloc unique vs nœud par nœud)

Les deux techniques d'allocation dynamique étaient déjà connues.

La seconde — nœud par nœud — devient ici indispensable, et non plus seulement élégante.

Le nœud auto-référencé

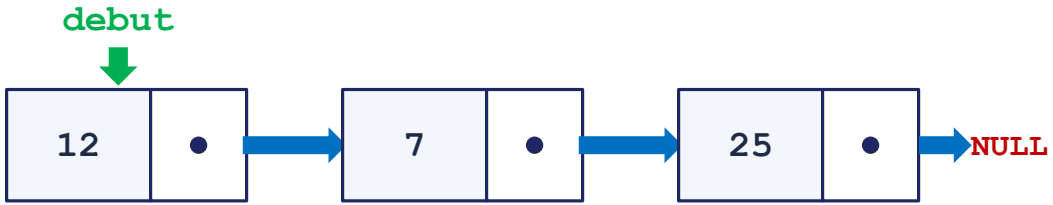
```
typedef struct noeud {
    int valeur;
    struct noeud *suivant;
} Noeud;
```



suivant, qui pointe vers un autre Noeud

Une structure qui se contient elle-même, par pointeur : c'est ce lien qui permet de chaîner les nœuds entre eux.

La liste : un seul repère, et un prix à payer




```
Noeud *debut = NULL;    /* liste vide */
```

Le prix à payer : contrairement à un tableau, pas d'accès direct par indice.

Pour atteindre un élément, il faut parcourir la liste depuis debut — accès séquentiel seulement.



Le fil narratif : le registre des inscrits

Le registre des inscrits d'une faculté :

Chaque inscription ajoute un nœud, chaque radiation en retire un. Le nombre d'étudiants varie librement au cours de l'année — le scénario où la liste chaînée est le choix naturel, pas un simple exercice de style.

Ce fil accompagnera les exemples de ce chapitre.

Déclaration

```
typedef struct noeud {
    int valeur;
    struct noeud *suivant;
} Noeud;

Noeud *debut = NULL;
```

debut est le repère vers le premier nœud.

Liste vide : **debut == NULL**.

Parcours et affichage

1) Partir du ***debut*** de la liste, avec un pointeur temporaire ***courant***.
2) Tant que ***courant*** n'est pas *NULL* :

   traiter la *valeur* du *nœud* ***courant***, puis avancer ***courant*** vers ***courant->suivant***.
3) S'arrêter quand ***courant*** devient *NULL* — la fin de la liste.

```
void afficherListe(void) {
    Noeud *courant = debut;
    while (courant != NULL) {
        printf("%d  ", courant->valeur);
        courant = courant->suivant;
    }
}
```

Insertion en tête

1) Créer un ***nouveau*** *nœud* (***malloc***), lui donner sa *valeur*.
2) Faire pointer *son* ***suivant*** vers l'*ancien* ***debut***.
3) ***debut*** devient le ***nouveau*** *nœud*.

```
void insererEnTete(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->suivant = debut;
    debut = nouveau;
}
```

Point de rigueur : relier le ***nouveau*** *nœud* à la suite avant de réaffecter **debut**. Inverser l'ordre casse le chaînage.

Insertion en fin

1) Créer un ***nouveau*** *nœud,* lui donner sa *valeur* et mettre *son* ***suivant*** à *NULL* (il sera le dernier).
2) Si la liste est vide (*debut == NULL*), le ***nouveau*** *nœud* devient directement ***debut***.
3) Sinon, parcourir la liste jusqu'au *dernier nœud* (celui dont ***suivant*** vaut *NULL*).
4) Faire pointer le ***suivant*** de ce *dernier nœud* vers le ***nouveau*** *nœud*.

```
void insererEnFin(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->suivant = NULL;
    if (debut == NULL) { debut = nouveau; return; }
    Noeud *courant = debut;
    while (courant->suivant != NULL) {
        courant = courant->suivant;
    }
    courant->suivant = nouveau;
}
```

Suppression en tête

1) Si la liste est vide, rien à faire.
2) Sinon, mémoriser le *nœud* ***debut*** actuel dans une variable temporaire.
3) Faire avancer ***debut*** vers ***debut->suivant***.
4) Libérer (***free***) l'*ancien nœud* mémorisé.

```
void supprimerEnTete(void) {
    if (debut == NULL) { return; }
    Noeud *ancienDebut = debut;
    debut = debut->suivant;
    free(ancienDebut);
}
```

Suppression d'un élément donné — le piège

Il faut garder, en même temps, le ***nœud précédent*** — pour pouvoir recoller la chaîne une fois le nœud visé retiré.

1) Si la liste est vide, rien à faire.
2) Si la valeur cherchée est dans le *nœud* ***debut***, cas particulier — traiter comme une *suppression en tête*.
3) Sinon, parcourir en gardant toujours deux pointeurs : ***precedent*** (*dernier nœud visité*) et ***courant*** (*nœud examiné*), jusqu'à trouver la valeur ou atteindre la fin.
4) Si trouvé, faire pointer ***precedent->suivant*** vers ***courant->suivant*** — le *nœud à supprimer* est ainsi « sauté ».
5) Libérer (***free***) le *nœud sauté*.

Suppression d'un élément donné — le code

```
void supprimerValeur(int valeur) {
    if (debut == NULL) { return; }
    if (debut->valeur == valeur) { supprimerEnTete(); return; }
    Noeud *precedent = debut;
    Noeud *courant = debut->suivant;
    while (courant != NULL && courant->valeur != valeur) {
        precedent = courant;
        courant = courant->suivant;
    }
    if (courant != NULL) {
        precedent->suivant = courant->suivant;
        free(courant);
    }
}
```

Une fois sur ***courant***, impossible de revenir en arrière dans une liste simplement chaînée — d'où la nécessité de ***precedent***.

Recherche d’un élément par sa valeur

1) Partir du ***debut*** en utilisant un pointeur temporaire ***courant***.
2) Tant qu'on n'a pas trouvé la ***valeur*** et qu'on n'a pas atteint la *fin*, avancer le pointeur ***courant***.
3) Retourner *vrai* (***1***) si trouvé, *faux* (***0***) sinon.

```
int rechercher(int valeur) {
    Noeud *courant = debut;
    while (courant != NULL) {
        if (courant->valeur == valeur) { return 1; }
        courant = courant->suivant;
    }
    return 0;
}
```

Détruire la liste — la dernière opération

1) Tant que la liste n'est pas vide, supprimer le *nœud en tête*.

```
void detruireListe(void) {
    while (debut != NULL) {
        supprimerEnTete();
    }
}
```

Réutilisation : ***supprimerEnTete*** est déjà définie, on se contente de la répéter jusqu'à ***debut == NULL***.

Rappel : Chaque ***malloc*** engage un ***free*** correspondant. Chaque nœud supprimé doit être libéré avant que son pointeur ne soit perdu — sans quoi c'est une fuite mémoire silencieuse.

Récapitulatif

- Déclaration : *Noeud auto-référencé* + ***debut = NULL***.
- Parcours : ***courant*** se déplace, ***debut*** ne change jamais.
- Insertion en tête (rapide) contre insertion en fin (parcours complet).
- Suppression en tête : mémoriser, avancer ***debut***, puis ***free***.
- Le piège central : garder ***precedent*** pour supprimer une valeur donnée.
- Recherche : même schéma que le parcours, avec un test.
- ***detruireListe*** : réutilise ***supprimerEnTete*** en boucle, jusqu'à la liste vide.
- Rappel transversal : chaque suppression de nœud engage un ***free***.

Exercice d’application

Le registre des inscrits d'une faculté est représenté par une liste simplement chaînée, ***debut*** étant une variable globale.

Chaque étudiant a un ***numero*** (entier), un ***nom*** (chaine) et un ***prenom*** (chaine).

1) Écrire ***inscrireEtudiant***, qui insère un nouvel étudiant en fin de registre.
2) Écrire ***radierEtudiant***, qui retire l'étudiant dont le numéro est donné, et libère la mémoire.
3) Dérouler l'inscription de 3 étudiants (101, 102, 103), puis la radiation de 102.

Question 1 — *inscrireEtudiant*

```c
typedef struct etudiant {
    int numero;
    char nom[30];
    char prenom[30];
    struct etudiant *suivant;
} NoeudEtudiant;

NoeudEtudiant *debut = NULL;

void inscrireEtudiant(int numero, char nom[], char prenom[]) {
    NoeudEtudiant *nouveau = malloc(sizeof(NoeudEtudiant));
    nouveau->numero = numero;
    strcpy(nouveau->nom, nom);
    strcpy(nouveau->prenom, prenom);

    nouveau->suivant = NULL;
    if (debut == NULL) { debut = nouveau; return; }
    NoeudEtudiant *courant = debut;
    while (courant->suivant != NULL) {
            courant = courant->suivant;
    }
    courant->suivant = nouveau;
}
```

Question 2 — *radierEtudiant*

```
void radierEtudiant(int numero) {
    if (debut == NULL) { return; }
    if (debut->numero == numero) {
        NoeudEtudiant *ancienDebut = debut;
        debut = debut->suivant;
        free(ancienDebut);
        return;
    }
    NoeudEtudiant *precedent = debut;
    NoeudEtudiant *courant = debut->suivant;
    while (courant != NULL && courant->numero != numero) {
        precedent = courant;
        courant = courant->suivant;
    }
    if (courant != NULL) {
        precedent->suivant = courant->suivant;
        free(courant);
    }
}
```

Question 3 — la trace

Étape	Action	État de la liste (numéros)
1	`inscrireEtudiant(101)`	`101`
2	`inscrireEtudiant(102)`	`101 → 102`
3	`inscrireEtudiant(103)`	`101 → 102 → 103`
4	`radierEtudiant(102)`	`101 → 103`

Etape 4: 102 était au milieu — ***precedent*** (101) et ***courant*** (102), ***precedent->suivant*** est reconnecté vers ***courant->suivant*** (103). ***free*** libère la place occupée par le noeud 102.

Exercice 1:

Soient les déclarations suivantes :

```
typedef struct noeud {
    int valeur;
    struct noeud *suivant;
} Noeud;

Noeud *debut = NULL;
```

1) Écrire une fonction ***insererApres (int apres, int valeur)*** qui insère une nouvelle ***valeur*** juste après le premier nœud contenant une valeur donnée (***apres***). Si cette valeur n'est pas trouvée, la liste reste inchangée.
2) Écrire une fonction ***insererAPosition(int position, int valeur)*** qui insère une nouvelle ***valeur*** juste après le nœud occupant une position donnée (***position)***, le premier nœud étant en position 1. Si cette position dépasse la taille de la liste, la liste reste inchangée.
3) Écrire une fonction ***supprimerValeurs(int valeur)*** qui supprime *tous* les nœuds contenant une ***valeur*** donnée.

Exercice 2:

La bibliothèque de l'école tient la liste (globale) des étudiants ayant un livre en retard, chaque nouveau retard étant ajouté **en tête** (le plus récent en premier).

1) Écrire ***void signalerRetard(int numero)***, qui insère un numéro d'étudiant en tête de la liste.
2) Écrire ***int nombreRetards(void)***, qui retourne le nombre d'étudiants actuellement en retard.
3) Dérouler ***signalerRetard*** pour les numéros **12**, **45**, **7** (dans cet ordre), puis donner l'état de la liste et le résultat de ***nombreRetards***.
4) Écrire ***void retourLivre(int numero)***, qui retire de la liste des retardataires le premier nœud correspondant à un ***numéro*** donné, et libère la mémoire correspondante.
5) En reprenant l'état final de de la question 3, dérouler ***retourLivre(45)*** et donner l'état final de la liste.
6) Que se passerait-il si ***retourLivre*** était appelée avec un numéro absent de la liste ?